

Développer une interface utilisateur Web mobile et mobile frontend avec Flutter

ABDELKERIM ALI ABBO

14 août 2025

Plan

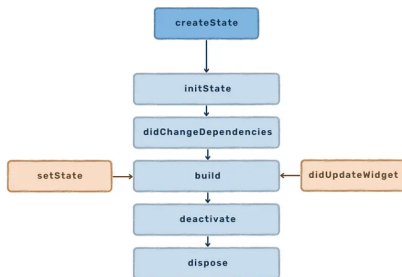
- 1 Rappel rapide sur Flutter Architecture (30 min)
- 2 Introduction aux appels HTTP en Flutter (45 min)
- 3 Introduction aux appels HTTP en Flutter (45 min)
- 4 Introduction aux appels HTTP en Flutter (45 min)
- 5 Introduction aux appels HTTP en Flutter (45 min)
- 6 Introduction aux appels HTTP en Flutter (45 min)
- 7 Atelier pratique : affichage de données d'une API (2h)

Architecture Flutter orientée composant (Widgets)

Flutter est composé de widgets. C'est le cœur du framework dont le slogan est « In Flutter everything is a Widget » ! En français, on pourrait dire qu'avec Flutter tout est widget.

Le Widget est donc l'élément central du SDK. Un widget se reconstruit lorsque son état change

Cycle de vie d'un Widget Stateful



- Création
- Mis à Jour
- Destruction

BuildContext

C'est un objet utilisé par Flutter pour fournir des informations sur l'emplacement d'un widget dans l'arborescence. Il représente le contexte de construction actuel dans lequel un widget est créé ou mis à jour. Il s'agit essentiellement d'une référence à l'emplacement d'un widget dans la hiérarchie des widgets.

setState

La méthode `setState()` de Flutter permet d'avertir le framework que l'état interne d'un `StatefulWidget` a changé et que le widget doit être reconstruit avec l'état mis à jour. Élément essentiel du modèle de programmation réactive de Flutter, elle permet de mettre à jour l'interface utilisateur en fonction des changements d'état.

immutabilité En Flutter, la mutabilité fait référence à la capacité d'un objet à être modifié après sa création. Un objet mutable peut changer de valeur. Flutter utilise ce concept pour gérer l'état de l'interface utilisateur,

avec des widgets immuables et des objets d'état potentiellement mutables.

Présentation de la bibliothèque http

Une bibliothèque HTTP est un ensemble de fonctions et de classes pré-écrites qui permettent aux développeurs d'envoyer des requêtes (comme des requêtes GET pour récupérer des données ou des requêtes POST pour envoyer des données) à des serveurs web et de traiter les réponses, sans avoir à implémenter manuellement tout le protocole HTTP.

API REST

Les API REST (Representational State Transfer) constituent un ensemble de principes architecturaux pour la conception d'applications réseau. Elles exploitent les méthodes et protocoles HTTP standard pour la communication entre clients et serveurs.

GET

permet de récupérer des données d'une ressource spécifiée. Les requêtes GET ne devraient pas avoir d'effets secondaires sur le serveur et sont considérées comme « sûres » et « idempotentes »

POST

permet d'envoyer des données au serveur pour créer une nouvelle ressource. Les requêtes POST ne sont pas idempotentes, ce qui signifie que plusieurs requêtes identiques peuvent entraîner la création de plusieurs ressources.

JSON

Format d'échange de données léger, couramment utilisé pour la transmission de données dans les API REST. Il est lisible par l'homme et facilement analysable par les machines. JSON représente les données sous forme de paires clé-valeur et de listes ordonnées de valeurs.

Headers

Les en-têtes HTTP sont des paires clé-valeur qui transmettent les métadonnées de la requête ou de la réponse. Ils fournissent les informations nécessaires au serveur ou au client pour traiter le message.

Utilisation de Future, async et await

async et **await** sont utilisés pour simplifier la gestion des opérations asynchrones, en particulier avec les **Future**. **async** marque une fonction comme asynchrone, lui permettant de retourner un **Future**. **await** est utilisé à l'intérieur d'une fonction **async** pour attendre la résolution d'un **Future** sans bloquer l'exécution du reste de l'application.

Parsing de données JSON avec jsonDecode()

En Flutter, `jsonDecode()` est une fonction du package `dart:convert` qui permet de transformer une chaîne de caractères JSON en un objet Dart, généralement une `Map<String, dynamic>` ou une `List<dynamic>`. C'est une étape cruciale pour travailler avec des données JSON reçues d'une API ou chargées depuis un fichier.

Créer une app Flutter qui récupère des données depuis une API publique Voici un lien vers mon depot github : [Cliquer ici](#)